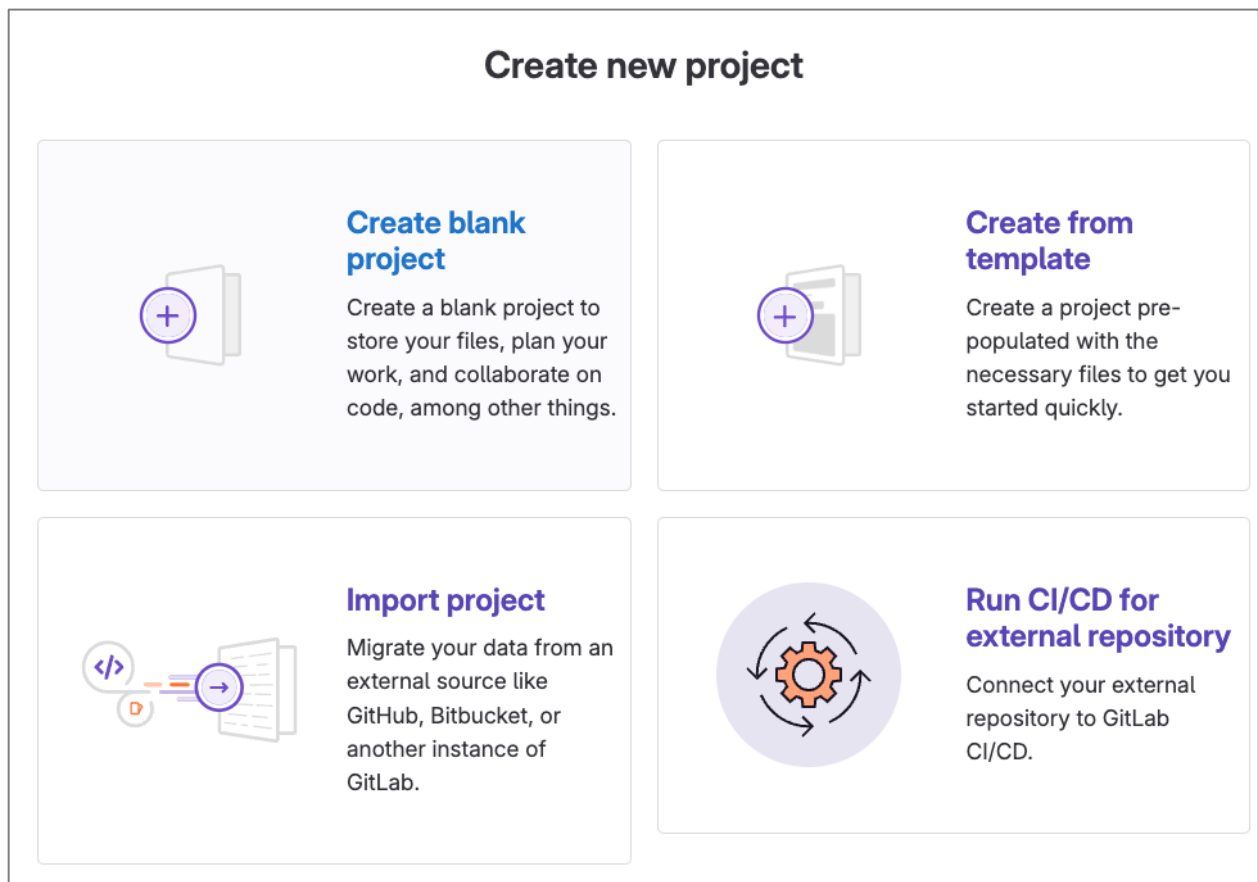
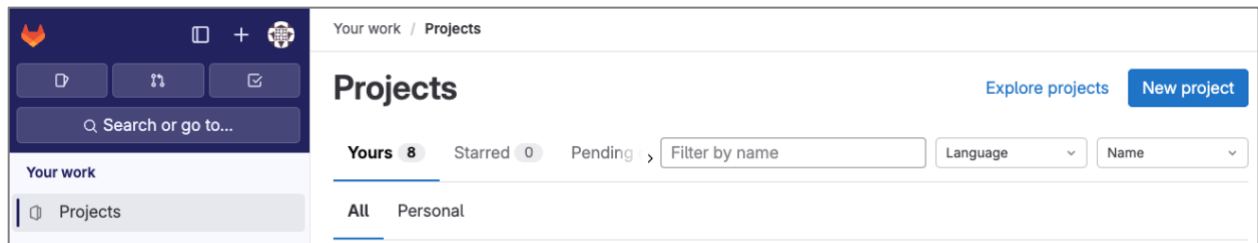


Git (gitlab with screenshots)

Step 1. Create your project

Group's leader does this!





Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

Project name

Must start with a lowercase or uppercase letter, digit, emoji, or underscore. Can also contain dots, pluses, dashes, or spaces.

Project URL

Project slug

Want to organize several dependent projects under the same namespace? [Create a group](#).

Project deployment target (optional)

Visibility Level

☒ Private

Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.

☐ Public

The project can be accessed without any authentication.

Project Configuration

☒ Initialize repository with a README

Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.

☐ Enable Static Application Security Testing (SAST)

Analyze your source code for known security vulnerabilities. [Learn more](#).

Step 2. Clone the project to your local machine

Get the URL

Project 'awesome2024' was successfully created.

A awesome2024

main awesome2024 / +

History Find file Edit Code

 **Initial commit**
Nguyễn Thanh Tâm (FE FPTU HN) authored just now

Name	Last commit
 README.md	Initial commit

 README.md

awesome2024

Getting started

To make it easy for you to get started with GitLab, here's a list of recommended next steps:

Already a pro? Just edit this README.md and make it your own. Want to learn more?
[View the project README](#) [at the bottom!](#)

Add your files

☐ Create or upload files

Clone with SSH

git@gitlab.com:tamnt122/aweson 

Clone with HTTPS

https://gitlab.com/tamnt122/av 

Open in your IDE

Visual Studio Code (SSH)
Visual Studio Code (HTTPS)
IntelliJ IDEA (SSH)
IntelliJ IDEA (HTTPS)

Download source code

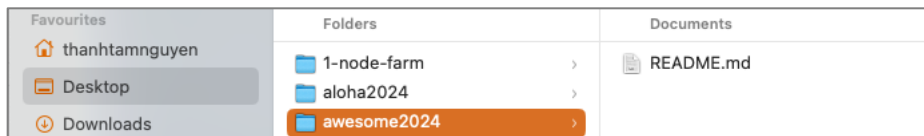
zip
tar.gz
tar.bz2
tar

Copy URL

Clone the project `git clone project_url`

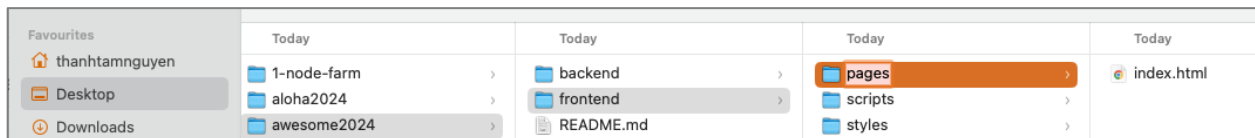
```
Apple > ~/Desktop > git main ?31 03:35:40 PM
git clone https://gitlab.com/tamnt122/awesome2024.git
Cloning into 'awesome2024'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
```

The project is cloned to your local machine



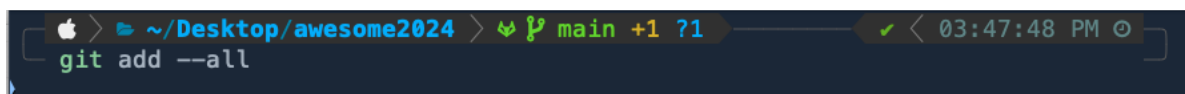
Step 3. Create your project skeleton (locally)

It is recommended that this step should be done by one member.

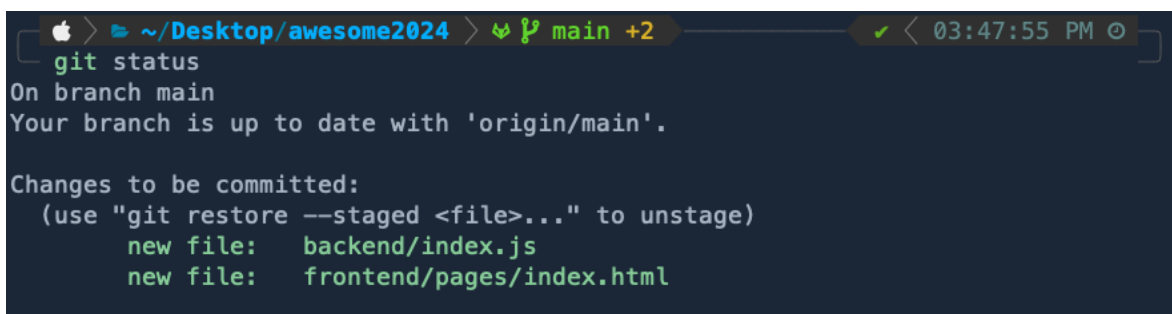


Step 4. Push it to Git

Stage the files



Check the files' status



Commit the files. Important: **Make the message informative!**

```
Apple > ~/Desktop/awesome2024 > main +2 03:47:57 PM
git commit -m "adding project skeleton files"
[main 758fab9] adding project skeleton files
2 files changed, 88 insertions(+)
create mode 100644 backend/index.js
create mode 100644 frontend/pages/index.html
```

Your commit will show up on Gitlab

A

awesome2024

main

awesome2024 /

+

History

Find file

Edit

Code



adding project skeleton files

758fab99



Nguyễn Thanh Tâm (FE FPTU HN) authored 1 minute ago

Name	Last commit	Last update
backend	adding project skeleton files	1 minute ago
frontend/pages	adding project skeleton files	1 minute ago
README.md	Initial commit	17 minutes ago

Step 5. Branching

Option 1. Do it locally

Create a branch locally from the current branch (in this case, it's main)

```
Apple > ~/Desktop/awesome2024 > main 03:53:53 PM
git branch member1
```

Make sure it's there

```
Apple > ~/Desktop/awesome2024 > main 03:54:13 PM
git branch -a
```

```
* main
member1
remotes/origin/HEAD -> origin/main
remotes/origin/main
(END)
```

Checkout the branch

```
Apple > ~/Desktop/awesome2024 > git main ?1 04:01:48 PM
git checkout member1
Switched to branch 'member1'
```

Create a new file

Today	Today	Today
- backend - frontend - README.md	- pages - scripts - styles	- contact-us.html - index.html

Change current directory to the one with newly created file, stage the file, then commit it

```
Apple > ~/Desktop/awesome2024 > git member1 ?1 04:02:11 PM
cd frontend/pages

Apple > ~/Desktop/awesome2024/f/pages > git member1 ?1
git add contact-us.html

Apple > ~/Desktop/awesome2024/f/pages > git member1 +1
git commit -m "adding contact-us.html page"
[member1 160fb49] adding contact-us.html page
1 file changed, 16 insertions(+)
create mode 100644 frontend/pages/contact-us.html
```

Push push push. Important: Only use this command to push a newly created branch along with its files

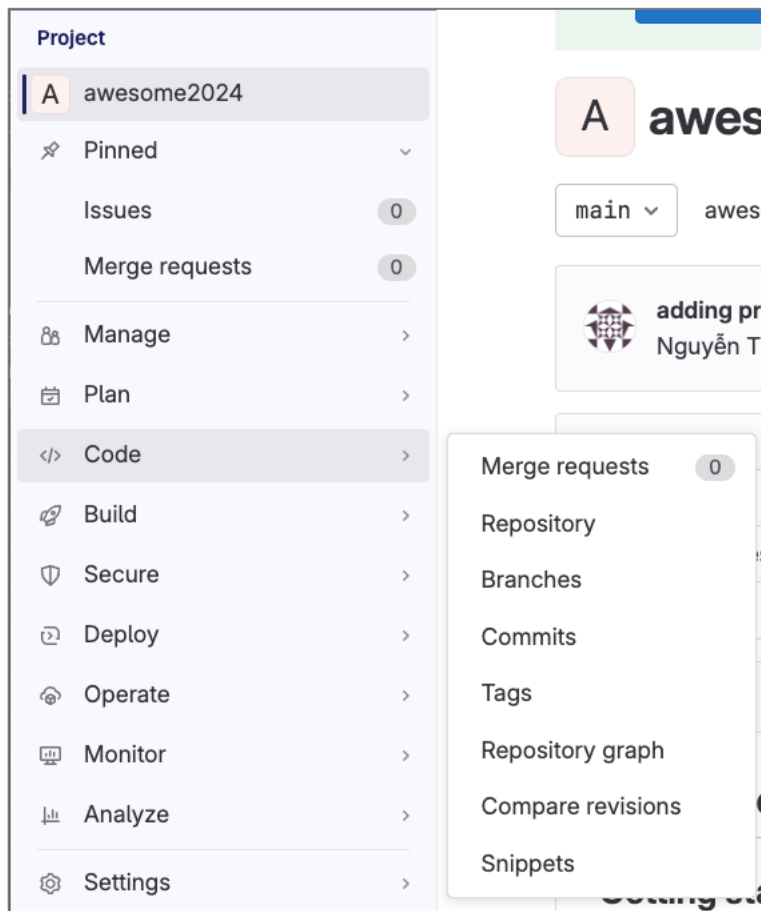
```

[ Apple > ~/Desktop/awesome2024/f/pages > git member1 04:06:24 PM ]
git push -u origin member1
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 645 bytes | 645.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for member1, visit:
remote:   https://gitlab.com/tamnt122/awesome2024/-/merge_requests/new?merge_req
uest%5Bsource_branch%5D=member1
remote:
To https://gitlab.com/tamnt122/awesome2024.git
* [new branch]      member1 -> member1
branch 'member1' set up to track 'origin/member1'.

```

Option 2. Do it on Gitlab

Code → Branches



Create a new branch

New Branch

Branch name

Create from

main

Existing branch name, tag, or commit SHA

Create branchCancel

On your local machine, fetch the change on the remote repo

```
Apple > ~/Desktop/awesome2024 > member1 04:14:29 PM
git fetch
From https://gitlab.com/tamnt122/awesome2024
* [new branch]      member2      -> origin/member2
```

Now we can see the new branch

```
Apple > ~/Desktop/awesome2024 > main 04:15:13 PM
git branch -a

* main
  member1
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
  remotes/origin/member1
  remotes/origin/member2
(END)
```

Salmon = remote

Gray = the last branches that you just checked out

Green = current branch you're in

Same steps as Option 1:

Checkout the branch you want to work on

→ CUD files as needed

→ Stage file(s)

→ Commit them (with a proper message)

→ Push git push

```
Apple > ~/Desktop/awesome2024 > member2 ?1 04:24:04 PM
cd frontend/styles

Apple > ~/Desktop/awesome2024/f/styles > member2 ?1
git add sample-style1.css

Apple > ~/Desktop/awesome2024/f/styles > member2 +1
git status
On branch member2
Your branch is up to date with 'origin/member2'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   sample-style1.css

Apple > ~/Desktop/awesome2024/f/styles > member2 +1
git commit -m "adding the first style template"
[member2 a9b85bd] adding the first style template
1 file changed, 31 insertions(+)
create mode 100644 frontend/styles/sample-style1.css

Apple > ~/Desktop/awesome2024/f/styles > member2 +1
git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 566 bytes | 566.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for member2, visit:
remote:   https://gitlab.com/tamnt122/awesome2024/-/merge_requests/new?merge_req
uest%5Bsource_branch%5D=member2
remote:
To https://gitlab.com/tamnt122/awesome2024.git
758fab9..a9b85bd member2 -> member2
```

Step 6. Merge to main

Create a merge request

✓

You pushed to **member2** 2 minutes ago

×

Create merge request

main

awesome2024 / frontend / +

History

Find file

Edit

Code

 **adding project skeleton files**

Nguyễn Thanh Tâm (FE FPTU HN) authored 38 minutes ago

758fab99



Name	Last commit	Last update
..		
 pages	adding project skeleton files	38 minutes ago

New merge request

From **member2** into **main** [Change branches](#)

Title (required)

adding the first style template

☐ Mark as draft
Drafts cannot be merged until marked ready.

Description

Preview | **B** *I*   |                                        

Describe the goal of the changes and what reviewers should be aware of.

Switch to rich text editing

Add [description templates](#) to help your contributors to communicate effectively!

Assignee

Unassigned [Assign to me](#)

Reviewer

Unassigned

Milestone

Select milestone

Labels

Select label

Merge options

☐ Delete source branch when merge request is accepted.

☐ Squash commits when merge request is accepted. 

Create merge request Cancel

Merge the branch to main

adding the first style template

Open **Nguyễn Thanh Tâm (FE FPTU HN)** requested to merge **member2** into **main** just now

Overview 0 Commits - Pipelines 0 Changes -

0 0

8 Approve Approval is optional

Ready to merge!

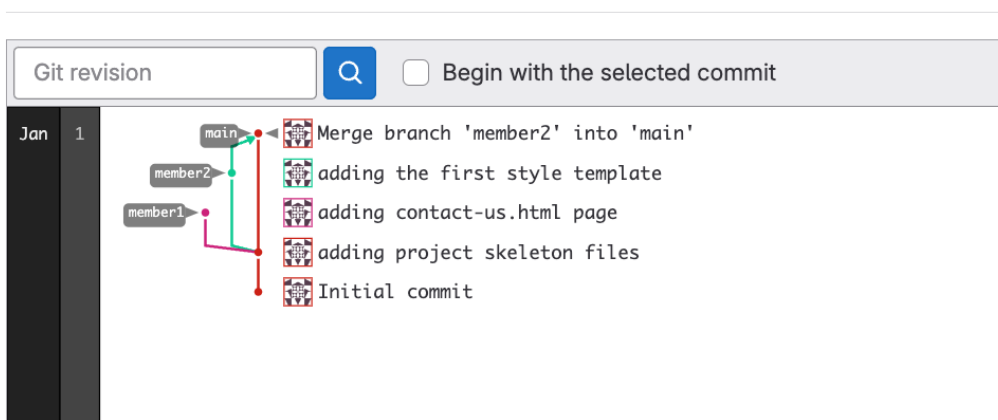
☐ Delete source branch ☐ Squash commits ☐ Edit commit message

1 commit and 1 merge commit will be added to **main**.

Activity All activity

Create a new merge request manually

In this example, member1 branch is still hanging around



In this case, we can create a new merge request

Nguyễn Thanh Tâm (FE FPTU HN) / awesome2024 / Merge requests

Open 0

Merged 1

Closed 0

All 1

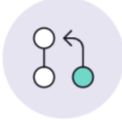
Bulk edit

New merge request

Recent searches

Search or filter results...

Created date



Merge requests are a place to propose changes you've made to a project and discuss those changes with others

Interested parties can even contribute by pushing commits if they want to.

New merge request

New merge request

[Email a new merge request to this project](#)

New merge request

Source branch

tamnt122/awesome2024

member1

Target branch

tamnt122/awesome2024

main

 adding contact-us.html page

Nguyễn Thanh Tâm (FE FPTU HN) authored

Jan 01, 2024

160fb490

 Merge branch 'member2' into 'main' ...

Nguyễn Thanh Tâm (FE FPTU HN) authored

Jan 01, 2024

1c44fe07

Compare branches and continue

New merge request

From `member1` into `main` [Change branches](#)

Title (required)

adding contact-us.html page

☐ Mark as draft

Drafts cannot be merged until marked ready.

Description

Preview

B

I

U

Code

Link

Table

List

Order

Quote

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Table

Describe the goal of the changes and what reviewers should be aware of.

Switch to rich text editing

Add [description templates](#) to help your contributors to communicate effectively!

Assignee

Unassigned

[Assign to me](#)

Reviewer

Unassigned

Milestone

Select milestone

Labels

Select label

Merge options

☐ Delete source branch when merge request is accepted.

☐ Squash commits when merge request is accepted. [?](#)

Create merge request

Cancel

Step 7: Merge from main

At the end of each iteration (after every member has merged their branch into main), each member needs to merge main into their branch so that you get the most updated code.

```
Apple > ~/Desktop/awesome2024/f/pages > main ?1 04:55:08 PM
git checkout member1
Switched to branch 'member1'
Your branch is up to date with 'origin/member1'.

Apple > ~/Desktop/awesome2024/f/pages > member1 ?1
git merge main
Updating 160fb49..dc7898b
Fast-forward
 frontend/styles/sample-style1.css | 31 ++++++
 1 file changed, 31 insertions(+)
 create mode 100644 frontend/styles/sample-style1.css
```

Extra

Step 2. Get git initialised.

S1-1. Check if git is installed.

```
git --version
```

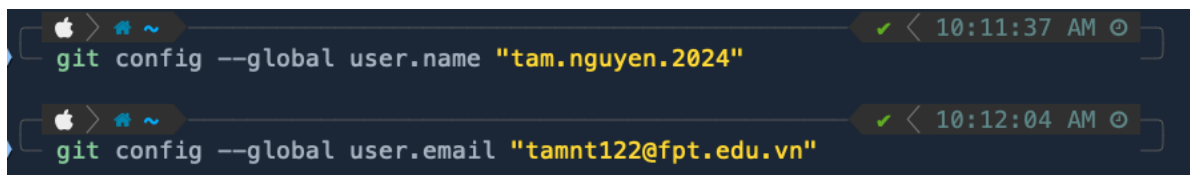
Git can be downloaded here: <https://www.git-scm.com/>

S1-2. Configure git.

```
git config --global user.name "your-username"
```

then

```
git config --global user.email "your-fpt-email"
```

A terminal window with a dark background. The first command is 'git config --global user.name "tam.nguyen.2024"' and the second is 'git config --global user.email "tamnt122@fpt.edu.vn"'. Both commands are preceded by a prompt 'apple >' and a home icon. To the right of each command is a green checkmark, a left arrow, and a timestamp: '10:11:37 AM' for the first and '10:12:04 AM' for the second. The terminal output shows the commands being executed successfully.

```
apple > git config --global user.name "tam.nguyen.2024" ✓ < 10:11:37 AM  
apple > git config --global user.email "tamnt122@fpt.edu.vn" ✓ < 10:12:04 AM
```

S1-3. Make a working directory.

Do this as you like!

Let's say I've created a folder on desktop named "1-node-farm".

Get in there!

A terminal window with a dark background. The command 'cd Desktop/1-node-farm' is entered. It is preceded by a prompt 'apple >' and a home icon. To the right is a green checkmark, a left arrow, and a timestamp: '10:13:05 AM'.

```
apple > cd Desktop/1-node-farm ✓ < 10:13:05 AM
```

S1-4. Initialize git.

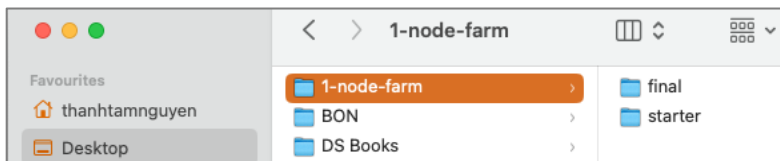
```
git init
```

```
Apple > ~/Desktop/1-node-farm 10:13:49 AM
git init
Initialized empty Git repository in /Users/thanhtamnguyen/Desktop/1-node-farm/.git/
```

- You just created your first Git Repository!
- **Note:** Git now knows that it should watch the folder you initiated it on → Git creates a hidden folder to keep track of changes.

Step 2. Adding files

S2-1. Add folders & files as needed for your project.



S2-2. List those (just to be sure!).

```
ls
```

```
Apple > ~/Desktop/1-node-farm git main ?2 11:00:52 AM
ls
final starter
```

S2-3. Check git's status to see the folders are part of the repository.

```
git status
```

```
Apple > ~/Desktop/1-node-farm git main ?2 11:00:54 AM
git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    final/
    starter/

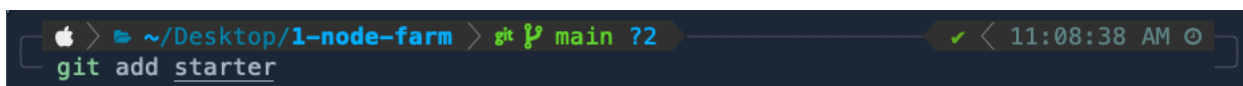
nothing added to commit but untracked files present (use "git add" to track)
```

Step 3. Staging files

Staged files are files that are ready to be **committed** to the repository you are working on.

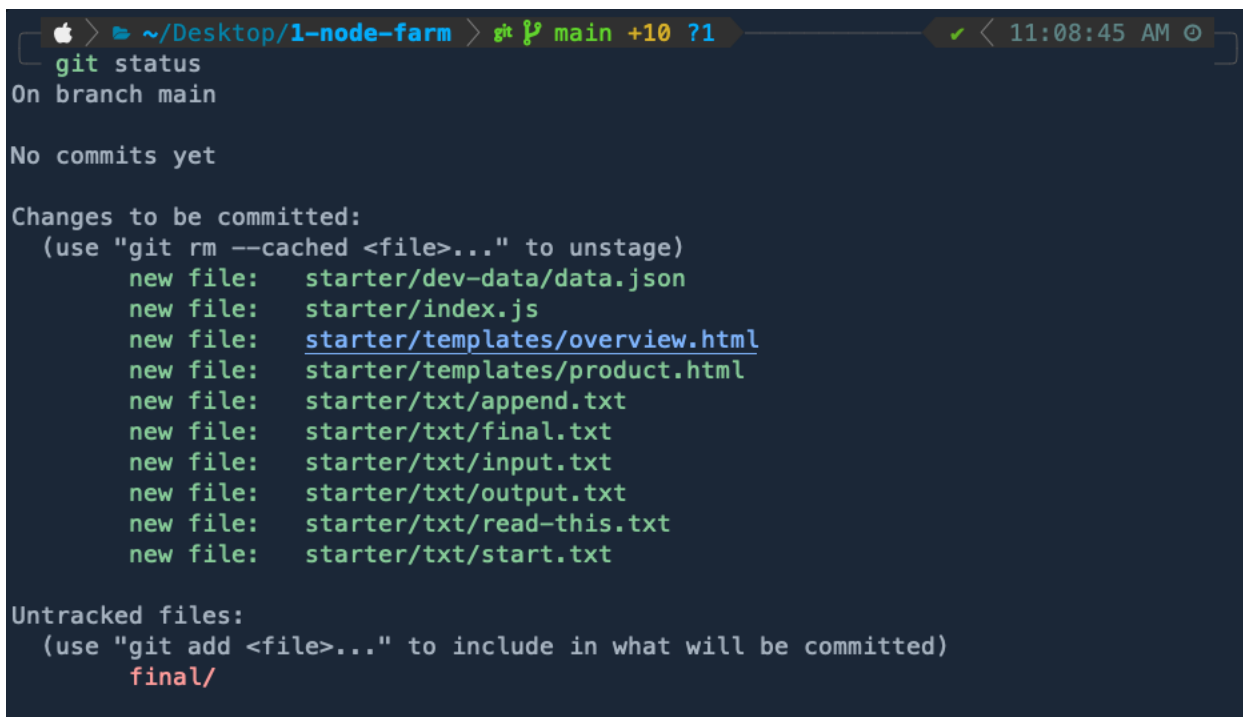
To add a file/folder to the Staging environment

```
git add starter
```



```
Apple > ~/Desktop/1-node-farm > git main ?2  
git add starter
```

Check the git status:



```
Apple > ~/Desktop/1-node-farm > git main +10 ?1  
git status  
On branch main  
  
No commits yet  
  
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
    new file:   starter/dev-data/data.json  
    new file:   starter/index.js  
    new file:   starter/templates/overview.html  
    new file:   starter/templates/product.html  
    new file:   starter/txt/append.txt  
    new file:   starter/txt/final.txt  
    new file:   starter/txt/input.txt  
    new file:   starter/txt/output.txt  
    new file:   starter/txt/read-this.txt  
    new file:   starter/txt/start.txt  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    final/
```

→ All files in starter folder are ready to be committed.

NB. We can stage all files/folder in the current directory to the Staging Environment as such:

```
git add --all
```

or

```
git add --A
```

Step 4. Committing files

